Ingenic®
Halley6 U-boot Development Manual

Author: System Software Department

Version: 1.0

Date: 2021.10.15

北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

Ingenic®

Halley6 U-boot Development Manual

Release history

Date	Revision	Change
2021.10.15	1.0	First release
2022.12.20	2.0	

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

Ingenic Semiconductor Co., Ltd.

Ingenic Headquarters, East Bldg. 14, Courtyard #10
Xibeiwang East Road, Haidian District, Beijing, China,
Tel: 86-10-56345000
Fax:86-10-56345001
Http: //www.ingenic.com

CATALOG

Key words	U-Boot Configuration Compilation Common Commands Startup Parameters
abstract	This document is applicable to the halley6 development board launched by Innocent Company. The purpose is to help developers familiarize themselves with the configuration, compilation method, commonly used commands, and the interface with the linux kernel supported by the u-boot project supporting the development board.
abbreviation	
References	

1 u-boot Introduction

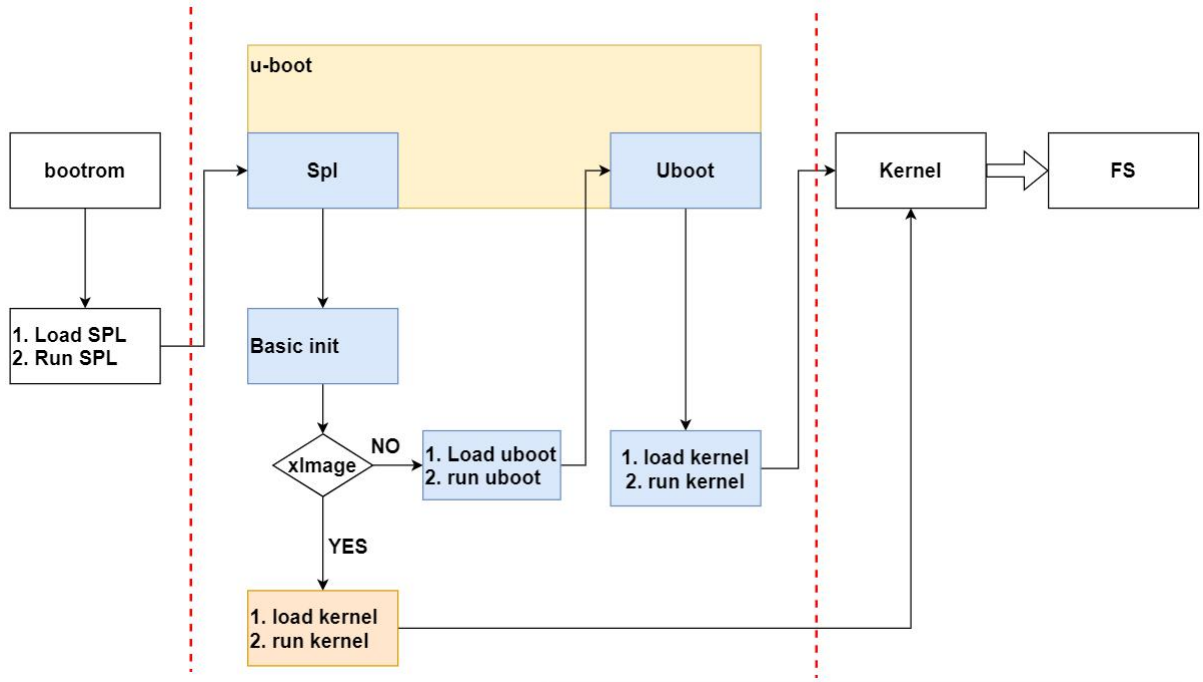
1.1 u-boot Function

U-Boot is an open source bootloader for embedded platforms, which can boot a variety of embedded operating systems including the linux kernel, supports multiple processor architectures such as mips, arm, powerpc, etc. It provides serial It supports various download methods such as port, Ethernet, etc., provides functions such as NOR and NAND flash memory and environment variable management, supports network protocol stack, JFFS2/EXT2/FAT file system, and also supports various device drivers such as MMC/SD card, USB device, LCD driver, etc.

Note: The Nor mentioned in the text has no special instructions, all refer to SPI Nor Flash. Nand No special instructions, all refer to SPI Nand Flash.

1.2 u-boot Boot process

The u-boot boot process of Halley6 platform is shown in the figure below:



As can be seen from the above figure, there are two ways to guide:

1. bootrom ---> spl ---> u-boot ---> kernel ---> rootfs,This method requires u-boot to boot the kernel.

2. bootrom ---> spl ---> kernel ---> rootfs,This method does not require u-boot, and the kernel is directly guided by spl.

1.3 u-boot Introduction to the source code directory

api	Store the interface functions provided by uboot
arch	Architecture related files
board	Board-level related files
common	General code, mainly based on various commands
disk	Disk partition related code
doc	Introductory Documentation
drivers	driver related code
examples	sample program
fs	File system code, supporting common embedded file systems
include	head File
lib	Common library files
nand_spl	nand flash startup related code
net	network related code
post	Power On Self Test, power-on self-test procedure
spl	spl related code
test	test code
tools	Auxiliary tool for compiling and checking uboot object files

2 u-boot configure

On the current halley6 platform, the files that determine the u-boot configuration are mainly:

1. **u-boot/boards.cfg**
2. **u-boot/include/configs/halley6.h**

The two files work together to determine the configuration of the final generated burn image.

2.1 boards.cfg file description

boards.cfg is the most basic configuration file in u-boot, used to distinguish different board-level configurations from different manufacturers. Each line in this file represents a specific board-level configuration with the same format, as shown in the following table:

Target	ARCH	CPU	Board name	Vendor	SoC	Options
target name	schema name	processor name	board name	Trade Names	Soc Names	Additional options

Each of these configuration items is described as follows:

1. Target:as a sign of this configuration ;

2. ARCH、CPU、Board name、Vendor、SoC:It is mainly used to select the code directory when compiling the Makefile. For example, there are many configuration files in the u-boot/include/configs directory. The selection of halley6.h is determined by the Board name.

3. Options:The main difference between each configuration, u-boot prefixes each field in the additional options with "CONFIG_" and exports it as a globally available macro definition. These macro definitions will further determine other configurations. Currently, the main is the configuration in halley6.h.

2.1.1 Default configuration

halley6 supports a total of six default compilation configurations, as shown in the following table:

halley6_uImage_sfc_nand	**This configuration supports booting the linux kernel image uImage from nand flash** CONFIG_SPL_SFC_NAND:sfc nand flash Related configuration CONFIG_MTD_SFCNAND:sfc nand flash driver build configuration CONFIG_SPL_PARAMS_FIXER:Change clock frequency
halley6_uImage_sfc_nor	**This configuration supports booting the linux kernel image uImage from nor flash** CONFIG_SPL_SFC_NOR:sfc nor flash Related configuration CONFIG_ENV_IS_IN_SFC:Environment variables store related configuration

	CONFIG_MTD_SFCNOR:sfc nor flash Driver compilation related configuration CONFIG_SPL_PARAMS_FIXER:Change clock frequency
halley6_uImage_msc1	**This configuration supports booting the linux kernel image uImage from emmc** CONFIG_SPL_JZMMC_SUPPORT:MMC related configuration in spl CONFIG_ENV_IS_IN_MMC:Environment variable related configuration CONFIG_GPT_CREATOR: Partition related configuration CONFIG_JZ_MMC_MSC1:MSC controller related configuration CONFIG_SPL_PARAMS_FIXER:Change clock frequency
halley6_xImage_sfc_nand	**This configuration supports booting the linux kernel image xImage from nand flash** CONFIG_SPL_SFC_NAND:sfc nand flash Related configuration CONFIG_MTD_SFCNAND:sfc nand flash Driver compilation related configuration CONFIG_SPL_OS_BOOT: spl directly starts the linux kernel image related configuration CONFIG_SPL_PARAMS_FIXER:Change clock frequency
halley6_xImage_sfc_nor	**This configuration supports booting the linux kernel image xImage from nor flash** CONFIG_SPL_SFC_NOR：sfc nor flash Related configuration CONFIG_MTD_SFCNOR: sfc nor flash Driver compilation related configuration CONFIG_SPL_OS_BOOT: spl directly starts the linux kernel image related configuration CONFIG_ENV_IS_IN_SFC:Environment variable related configuration CONFIG_SPL_PARAMS_FIXER:Change clock frequency
halley6_xImage_msc1_ota	**This configuration supports booting the linux kernel image xImage from emmc** CONFIG_SPL_JZMMC_SUPPORT:MMC related configuration in spl CONFIG_ENV_IS_IN_MMC:Environment variable related configuration CONFIG_GPT_CREATOR: Partition related configuration CONFIG_JZ_MMC_MSC1:MSC controller related configuration CONFIG_SPL_PARAMS_FIXER:Change clock frequency CONFIG_OTA_VERSION30:ota upgrade related configuration

2.1.2 Custom configuration

Corresponding fields can be added or deleted in each board-level configuration or a new board-level configuration can be added according to actual needs.

The following is an example of adding a new board-level configuration **halley6_uImage_msc0**:

1. Start a new line in the boards.cfg file. According to the format requirements of the file, first determine that the first configuration item is halley6_uImage_msc0, and the second to sixth configuration items can refer to the existing board-level configuration, which are generally relatively fixed. of ;

2. It is mainly the configuration of additional options. If you are not familiar with the overall configuration of u-boot on the Halley6 platform, you can refer to the existing configuration first. For example, you can refer to halley6_uImage_msc1 here, and change JZ_MMC_MSC1 to JZ_MMC_MSC0, which will select the configuration related to MSC0. The others can be kept the same ;

3. If you want to modify the additional options in more detail, it is recommended to read the document as a whole, especially the introduction of the configuration in Chapter 2. After understanding the overall configuration of the u-boot of the Halley6 platform, it will be easier to modify it.

2.2 halley6.h configuration file description

halley6.h is the configuration file for halley6 platform core in u-boot. The configuration file mainly contains the following contents:

- System clock configuration
- Cache size configuration
- DDR configure
- SFC/MSC/GMAC/LCD Other driver related configuration
- u-boot built-in command

2.2.1 System Clock Configuration

2.2.1.1 Default Configuration

The current system clock configuration is as follows:

```
#define CONFIG_SYS_APLL_FREQ          1300000000
#define CONFIG_SYS_MPLL_FREQ          1200000000
#define CONFIG_SYS_EPLL_FREQ          200000000
#define CONFIG_CPU_SEL_PLL            APLL
#define CONFIG_DDR_SEL_PLL            MPLL
#define CONFIG_SYS_CPU_FREQ           1300000000
#define CONFIG_SYS_MEM_FREQ           400000000

#define CONFIG_SYS_EXTAL              24000000
```

```
#define CONFIG_SYS_HZ                   1000
```

2.2.1.2 Custom configuration

1. PLL phase-locked loop frequency configuration, which can be configured as needed

```
#define CONFIG_SYS_APLL_FREQ 1300000000 /*If APLL not use mast be set 0*/
#define CONFIG_SYS_MPLL_FREQ 1200000000 /*If MPLL not use mast be set 0*/
#define CONFIG_SYS_EPLL_FREQ 200000000 /*If MPLL not use mast be set 0*/
```

2. PLL selection of CPU and DDR, it is recommended to use the default configuration

```
#define CONFIG_CPU_SEL_PLL APLL
#define CONFIG_DDR_SEL_PLL MPLL
```

3. CPU, DDR frequency configuration, in which the CPU frequency needs to be a multiple of the selected PLL frequency, and supports up to 1.3GHz, and the DDR frequency also needs to be a multiple of the selected PLL frequency

```
#define CONFIG_SYS_CPU_FREQ 1300000000
#define CONFIG_SYS_MEM_FREQ 400000000
```

2.2.2 DDR Type configuration

2.2.2.1 Default configuration

The current default DDR parameters are as follows:

```
#define CONFIG_DDR_INNOPHY
#define CONFIG_DDR_DLL_OFF
#define CONFIG_DDR_PARAMS_CREATOR
#define CONFIG_DDR_HOST_CC
#define CONFIG_DDR_CS0 1 /* 1-connected, 0-disconnected */
#define CONFIG_DDR_CS1 0 /* 1-connected, 0-disconnected */
#define CONFIG_DDR_DW32 0 /* 1-32bit-width, 0-16bit-width */
/*#define CONFIG_DDR3_TSD34096M1333C9_E*/

#define CONFIG_DDR_PHY_IMPEDANCE 40
#define CONFIG_DDR_PHY_ODT_IMPEDANCE 120
/* #define CONFIG_FPGA_TEST */
/*#define CONFIG_DDR_AUTO_REFRESH_TEST*/

#define CONFIG_DDR_AUTO_SELF_REFRESH
#define CONFIG_DDR_AUTO_SELF_REFRESH_CNT 257
```

2.2.2.2 Custom configuration

The built-in DDR type of X1600 and X1600E chips is LPDDR2:

1. LPDDR2 configuration, if CONFIG_DDR_TYPE_LPDDR2 is defined, LPDDR2 related

configuration will be turned on:

```
/*#define CONFIG_DDR_TYPE_LPDDR2*/
#ifdef CONFIG_DDR_TYPE_LPDDR2
#define CONFIG_LPDDR2_M54D5121632A
#endif
```

2.2.3 SFC driver configuration

2.2.3.1 spl phase drive

Location: common/spl

```
├── spl_sfc_nand.c
├── spl_sfc_nand_v2.c
├── spl_sfc_nor.c
├── spl_sfc_nor_v2.c
```

2.2.3.2 uboot phase Driver files

Location: drivers/mtd/devices/jz_sfc_v2

```
├── jz_sfc_common.c
├── jz_sfc_common.h
├── jz_sfc_nand.c
├── jz_sfc_nor.c
├── jz_sfc_ops.c
├── Makefile
└── nand_device
    ├── ato_nand.c
    ├── dosilicon_nand.c
    ├── foresee_nand.c
    ├── gd_nand.c
    ├── mxic_nand.c
    ├── nand_common.c
    ├── nand_common.h
    ├── xtx_mid0b_nand.c
    ├── xtx_nand.c
    └── zetta_nand.c
```

2.2.3.3 Configuration Instructions

1. GPIO pin function configuration of sfc controller

```
/* sfc gpio */
CONFIG_SPL_SFC_SUPPORT
```

2. sfc nand flash configuration, if **CONFIG_SPL_SFC_NAND** is defined, the nand related

configuration will be opened:

```
/* sfc nand config */
#ifdef CONFIG_SPL_SFC_NAND
#define CONFIG_SFC_NAND_RATE 100000000 /* value <= 400000000(sfc 100Mhz)*/
#define CONFIG_SFC_QUAD
#define CONFIG_SPI_SPL_CHECK
#define CONFIG_SPIFLASH_PART_OFFSET 0x5800
#define CONFIG_SPI_NAND_BPP (2048 +64) /*Bytes Per Page*/
#define CONFIG_SPI_NAND_PPB (64) /*Page Per Block*/
#define CONFIG_JZ_SFC
#define CONFIG_CMD_SFCNAND
#define CONFIG_CMD_NAND
#define CONFIG_SYS_MAX_NAND_DEVICE 1
#define CONFIG_SYS_NAND_BASE 0xb3441000
#define CONFIG_SYS_MAXARGS 16
/*#define CONFIG_NAND_BUILTIN_PARAMS*/

/* sfc nand env config */
#define CONFIG_MTD_DEVICE
#define CONFIG_CMD_SAVEENV /* saveenv */
#define CONFIG_CMD_UBI
#define CONFIG_CMD_UBIFS
#define CONFIG_CMD_MTDPARTS
#define CONFIG_MTD_PARTITIONS
#define MTDIDS_DEFAULT "nand0:nand"
#define MTDPARTS_DEFAULT
    "mtdparts=nand:1M(boot),8M(kernel),40M(rootfs),-(data)"
#define CONFIG_SYS_NAND_BLOCK_SIZE (128 * 1024)
#endif
```

3. sfc nor flash configuration, if **CONFIG_SPL_SFC_NOR** is defined, the nor related configuration will be opened:

```
/* sfc nor config */
#ifdef CONFIG_SPL_SFC_NOR
#define CONFIG_JZ_SFC
#define CONFIG_CMD_SFC_NOR
#define CONFIG_JZ_SFC_NOR
#define CONFIG_SPI_SPL_CHECK
#define CONFIG_SFC_NOR_RATE 100000000 /* value <= 400000000(sfc 100Mhz)*/
#define CONFIG_SFC_QUAD
#define CONFIG_SPIFLASH_PART_OFFSET 0x5800
#define CONFIG_SPI_NORFLASH_PART_OFFSET 0x5874
#define CONFIG_NOR_MAJOR_VERSION_NUMBER 1
```

```
#define CONFIG_NOR_MINOR_VERSION_NUMBER 0
#define CONFIG_NOR_REVERSION_NUMBER 0
#define CONFIG_NOR_VERSION (CONFIG_NOR_MAJOR_VERSION_NUMBER |
    (CONFIG_NOR_MINOR_VERSION_NUMBER << 8) |
    (CONFIG_NOR_REVERSION_NUMBER <<16))
/*#define CONFIG_NOR_BUILTIN_PARAMS*/
#endif
```

2.2.3.4 Default Configuration

1. Boot with nand flash, **CONFIG_SPL_SFC_NAND** is exported in **halley6_uImage_sfc_nand** configuration and **halley6_xImage_sfc_nand** configuration in the boards.cfg file.

2. Boot with nor flash, and export **CONFIG_SPL_SFC_NOR** in the **halley6_uImage_sfc_nor** configuration of the boards.cfg file.

2.2.3.5 Custom Configuration

1. sfc controller gpio pin, which can be selected according to the hardware connection

```
/* sfc gpio */
CONFIG_SPL_SFC_SUPPORT
```

2. nand flash configure

a. sfc controller rate, note that the final rate is the result of dividing the macro by 4

```
#define CONFIG_SFC_NAND_RATE 100000000 /* value <= 400000000(sfc 100Mhz)*/
```

b. Data transmission line width, define this macro as 4 lines, otherwise it is 1 line

```
#define CONFIG_SFC_QUAD
```

c. Offset of nand flash partition parameters stored in nand flash

```
#define CONFIG_SPIFLASH_PART_OFFSET 0x5800
```

d. The nand flash page size and block size are determined according to the specific nand flash model

```
#define CONFIG_SPI_NAND_BPP (2048 +64) /*Bytes Per Page*/
#define CONFIG_SPI_NAND_PPB (64) /*Page Per Block*/
```

e. Whether to use built-in partition parameters, if defined, use, otherwise not use

```
/*#define CONFIG_NAND_BUILTIN_PARAMS*/
```

3. nor flash configure

a. SFC controller clock rate, note that the final rate is the result of dividing by 4 for this macro definition

```
#define CONFIG_SFC_NOR_RATE 100000000 /* value <= 400000000(sfc 100Mhz)*/
```

b. Data transmission line width, define this macro as 4 lines, otherwise it is 1 line

```
#define CONFIG_SFC_QUAD
```

c. The offset at which the nor flash partition parameters are stored in the nor flash

```
#define CONFIG_SPIFLASH_PART_OFFSET 0x5800
```

d. Offset of nor flash hardware parameters stored in nor flash

```
#define CONFIG_SPI_NORFLASH_PART_OFFSET 0x5874
```

e. Whether to use built-in partition parameters, if defined, use, otherwise not use

```
/*#define CONFIG_NOR_BUILTIN_PARAMS*/
```

2.2.4 MSC driver configuration

2.2.4.1 spl phase Driver files

Location: common/spl

├── spl_mmc.c

2.2.4.2 uboot phase Driver files

Location: drivers/mmc

└── jz_mmc.c

2.2.4.3 Configuration Instructions

1. MSC0 configuration, if CONFIG_JZ_MMC_MSC0 is defined, MSC0 related configuration will be opened:

```
#define CONFIG_GENERIC_MMC                  1
#define CONFIG_MMC                          1
#define CONFIG_JZ_MMC 1

#ifdef CONFIG_JZ_MMC_MSC0
#define CONFIG_JZ_MMC_SPLMSC 0
#define CONFIG_JZ_MMC_MSC0_PC_4BIT 1
#endif

#ifdef CONFIG_ENV_IS_IN_MMC
#define CONFIG_SYS_MMC_ENV_DEV              0
#define CONFIG_ENV_SIZE                     (32 << 10)
#define CONFIG_ENV_OFFSET                   (CONFIG_SYS_MONITOR_LEN +
CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR * 512)
#endif

#ifdef CONFIG_SPL_MMC_SUPPORT
#define CONFIG_SPL_SERIAL_SUPPORT
#define CONFIG_MMC_SPL_PARAMS
/*#define CONFIG_SPL_PARAMS_FIXER*/
#endif /* CONFIG_SPL_MMC_SUPPORT */
```

2. MSC1 configuration, if CONFIG_JZ_MMC_MSC1 is defined, MSC1 related configuration will be opened:

```
#define CONFIG_GENERIC_MMC                  1
#define CONFIG_MMC                          1
#define CONFIG_JZ_MMC 1

#ifdef CONFIG_JZ_MMC_MSC1
#define CONFIG_JZ_MMC_SPLMSC 1
#define CONFIG_JZ_MMC_MSC1_PD 1
#endif

#ifdef CONFIG_ENV_IS_IN_MMC
#define CONFIG_SYS_MMC_ENV_DEV              0
#define CONFIG_ENV_SIZE                     (32 << 10)
#define CONFIG_ENV_OFFSET                   (CONFIG_SYS_MONITOR_LEN +
CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR * 512)
#endif
#ifdef CONFIG_SPL_MMC_SUPPORT
#define CONFIG_SPL_SERIAL_SUPPORT
#define CONFIG_MMC_SPL_PARAMS
/*#define CONFIG_SPL_PARAMS_FIXER*/
#endif /* CONFIG_SPL_MMC_SUPPORT */
```

2.2.4.4 Default Configuration

Use the sd card to start, and export **CONFIG_JZ_MMC_MSC1** in the halley6_uImage_msc1 configuration of the boards.cfg file, that is, the MSC1 configuration is currently used by default.

2.2.4.5 Custom Configuration

In each configuration of MSC0 and MSC1, the following configurations can be customized:

1. Data transmission bits, MSC0 and MSC1 support up to 4 bits

```
#define CONFIG_SPL_JZ_MSC_BUS_4BIT
```

2. Whether to open DEBUG information

```
/*#define CONFIG_MMC_TRACE // only for DEBUG*/
```

2.2.4.6 Introduction to Partition Table

The u-boot image started by the SD card contains the partition table information. At the end of compiling the u-boot image, the following output information is displayed:

```
cat tools/ingenic-tools/mbr-gpt.bin u-boot-with-spl.bin > u-boot-with-spl-mbr-gpt.bin
```

It can be seen that mbr-gpt.bin is added in front of the final generated u-boot image, which is the partition data generated according to the GPT partition standard. The PC side recognizes the partition information, and does not need to use the partition tool to partition the SD card.

The partition data mbr-gpt.bin includes specific partition information. The partition information is introduced below. For the content related to MBR and GPT partition standards, please refer to 5.2MBR/GPT partition table.

Specific partition information is stored in u-boot/board/ingenic/halley6/partitions.tab:

```
property:
    disk_size = 4096m
    gpt_header_lba = 512
    custom_signature = 0

partition:
    #name       =   start,     size, fstype
    xboot       =      0m,       3m,
    boot        =      3m,       8m, EMPTY
    recovery    =     12m,      16m, EMPTY
    pretest     =     28m,      16m, EMPTY
    reserved    =     44m,      52m, EMPTY
    misc        =     96m,       4m, EMPTY
    cache       =    100m,     100m, LINUX_FS
    system      =    200m,    1800m, LINUX_FS
    data        =   2000m,   2048m, LINUX_FS

#fstype could be: LINUX_FS, FAT_FS, EMPTY
```

When we burn the image to the sd card, use the fdisk command on the PC side to see the following partition information:

```
Device          Start   End      Sector    Size Type
/dev/sdc1        6144   22527    16384     8M Microsoft basic data
/dev/sdc2       24576   57343    32768     16M Microsoft basic data
/dev/sdc3       57344   90111    32768     16M Microsoft basic data
/dev/sdc4       90112   196607   106496    52M Microsoft basic data
/dev/sdc5      196608   204799     8192     4M Microsoft basic data
/dev/sdc6      204800   409599   204800    100M Microsoft basic data
/dev/sdc7      409600 4095999 3686400    1.8G Microsoft basic data
/dev/sdc8     4096000 8290303 4194304     2G Microsoft basic data
```

The partition information is exactly the same as the content in the above partitions.tab file.

2.2.5 GMAC Network driver configuration

2.2.5.1 Driver files

Location: drivers/net

```
├──── jz_gmac_v12.c
├──── SynopGMAC_Dev.c
└──── SynopGMAC_Dev.h
```

2.2.5.2 Configuration Instructions

1. Ip address related

```
/* DEBUG ETHERNET */
#define CONFIG_SERVERIP 192.168.4.13
#define CONFIG_IPADDR 192.168.4.145
#define CONFIG_GATEWAYIP 192.168.4.1
#define CONFIG_NETMASK 255.255.255.0
#define CONFIG_ETHADDR 00:11:22:33:44:55
```

2. The network port working mode selection needs to be selected according to the actual connected network phy chip

```
#define GMAC_PHY_MII 0
#define GMAC_PHY_RMII 4
#define GMAC_PHY_GMII 0
#define GMAC_PHY_RGMII 1

#ifdef CONFIG_RGMII
#define CONFIG_NET_GMAC_PHY_MODE GMAC_PHY_RGMII
#else
#define CONFIG_NET_GMAC_PHY_MODE GMAC_PHY_RMII
#endif
```

3. Gmac Controller selection

```
#ifdef CONFIG_GMAC1
#define CONFIG_GAMAC_MODE_CTRL_ADDR 0xb00000e8
#define JZ_GMAC_BASE GMAC1_BASE
#define CONFIG_GMAC_CRLT_PORT GPIO_PORT_B
#define CONFIG_GMAC_CRLT_PORT_PINS (0xffff << 8)
#define CONFIG_GMAC_CRTL_PORT_INIT_FUNC GPIO_FUNC_3
#define CONFIG_GMAC_PHY_RESET GPIO_PB(16)
#define CONFIG_GMAC_TX_CLK_DELAY 0x3f
#define CONFIG_GMAC_RX_CLK_DELAY 0
#endif
#ifdef CONFIG_GMAC0
#define CONFIG_GAMAC_MODE_CTRL_ADDR 0xb00000e4
#define JZ_GMAC_BASE GMAC0_BASE
#define CONFIG_GMAC_CRLT_PORT GPIO_PORT_C
#define CONFIG_GMAC_CRLT_PORT_PINS (0x7fff << 1)
#define CONFIG_GMAC_CRTL_PORT_INIT_FUNC GPIO_FUNC_1
#define CONFIG_GMAC_PHY_RESET GPIO_PC(22)
```

```
#define CONFIG_GMAC_TX_CLK_DELAY 0x3f
#define CONFIG_GMAC_RX_CLK_DELAY 0
#endif

#define CONFIG_GMAC_CRTL_PORT_SET_FUNC GPIO_INPUT
#define CONFIG_GMAC_PHY_RESET_ENLEVEL 0
```

2.2.5.3 Default Configuration

1. The gmac0 controller is currently selected by default:

```
/* Select GMAC Controller */
/*#define CONFIG_GMAC0*/
```

2. The working mode of the RMII network port is currently selected by default:

```
/* Select GMAC Interface mode */
#define CONFIG_RMII
```

2.2.5.4 Custom Configuration

1. Currently, u-boot only supports one gmac controller to work at the same time. According to the actual hardware configuration, the controller can choose gmac0 or gmac1.

2. According to the actual speed needs, the working mode can choose RGMII or RMII, among which RGMII supports 10/100/1000 megabit bus interface speed, RMII supports 10 megabit and 100 megabit bus interface speed.

2.2.6 LCD driver configuration

2.2.6.1 Driver files

Location: drivers/video/jz_lcd

```
├── jz_lcd_v14.c
├── jz_mipi_dsi
├── lcd_panel
```

2.2.6.2 Configuration Instructions

If need to display the log in uboot phase need to include/configs/halley6.h open # define CONFIG_LCD this macro.

```
#ifdef CONFIG_LCD
#define LCD_BPP                 5 /* 4: 16BPP, 5: 24BPP. */
#define CONFIG_LCD_LOGO
#define CONFIG_LCD_ENABLE_RDMA_FB
```

2.2.6.3 Custom Configuration

If the customer wants to display their log during the uboot phase:

1. Add the displayed log image to the /tools/logos directory.
2. Modify tools/Makefile

```
ifneq ($(wildcard logos/$(VENDOR).bmp),)
LOGO_BMP ?= logos/$(VENDOR).bmp        #Change the.bmp file to the name of the log picture you want to display.
```

2.2.7 u-boot built-in Command configuration

2.2.7.1 Default Configuration

The following commands are currently supported by default:

```
/**
* Command configuration.
*/
#define CONFIG_CMD_BOOTD /* bootd */
#define CONFIG_CMD_CONSOLE /* coninfo */
#define CONFIG_CMD_DHCP /* DHCP support */
#define CONFIG_CMD_ECHO /* echo arguments */
#define CONFIG_CMD_EXT4 /* ext4 support */
#define CONFIG_CMD_FAT /* FAT support */
#define CONFIG_USE_XYZMODEM /* xyzModem */
#define CONFIG_CMD_LOAD /* serial load support */
#define CONFIG_CMD_LOADB /* loadb */
#define CONFIG_CMD_LOADS /* loads */
#define CONFIG_CMD_MEMORY /* md mm nm mw cp cmp crc base loop mtest */
#define CONFIG_CMD_MISC /* Misc functions like sleep etc*/
#define CONFIG_CMD_NET /* networking support */
#define CONFIG_CMD_PING
#define CONFIG_CMD_RUN /* run command in env variable */
#define CONFIG_CMD_SETGETDCR /* DCR support on 4xx */
#define CONFIG_CMD_SOURCE /* "source" command support */
#define CONFIG_CMD_GETTIME
#define CONFIG_CMD_GPIO
#define CONFIG_CMD_EXT2
#define CONFIG_CMD_EXT4
#define CONFIG_CMD_FAT
#define CONFIG_EFI_PARTITION
#define CONFIG_SOFT_BURNER

#define CONFIG_CMD_DDR_TEST /* DDR Test Command */
```

2.2.7.2 Custom Configuration

The following takes a simple hello command as an example to introduce how to customize the

command in u-boot.

1. In the common directory, create a new file cmd_hello.c and enter the following code:

```
#include<command.h>
#include<common.h>

static int do_hello(cmd_tbl_t *cmdtp, int flag, int argc, char * const argv[])
{
        printf("hello world! \n");
        return 0;
}

U_BOOT_CMD(
        hello,      1,      0,      do_hello,
        "short description. This is a string. \n",
        "long description. This is a string. \n"
);
```

In the above code, U_BOOT_CMD is the macro definition used by the u-boot new command. There are a total of 6 parameters. The details are as follows:

a. The first parameter: the name of the command
b. The second parameter: the number of parameters of the command
c. The third parameter: whether to repeat the command after pressing the enter key
d. The fourth parameter: the callback function corresponding to the command
e. The fifth parameter: short help information for the command
f. The sixth parameter: long help information for the command

2. Add compile options in common/Makefile:

```
COBJS-y += cmd_hello.o
```

3. Just recompile u-boot

2.2.8 Kernel parameter transfer instructions

Refer to the u-boot kernel parameter transfer instructions.

3 u-boot compile

There are two types of u-boot compilation, and the two features are as follows:

Compile method	Features
Compile the project top-level directory	Depends on the board-level configuration of Ingenic SDK. Suitable for development methods based on Ingenic SDK
u-boot source directory compilation	Directly follow the standard compilation method of u-boot, which is suitable for the configuration of u-boot that needs to be customized, and is separated from the development method of Ingenic SDK.

3.1 Project top-level directory compilation

3.1.1 Compile SFC Nand boot image

The boot image corresponds to the halley6_uImage_sfc_nand configuration.

1. Execute the following command in the top-level directory of the project to import the environment variables required for compiling the halley6 platform:

```
$ source build/envsetup.sh
```

2. Execute the lunch command to select the type of image to be compiled:

```
$ lunch
```

a. If it is halley6 v10 development board, choose among the following configurations as needed:

```
10. halley6.v10_nand_4.4.94-eng
11. halley6.v10_nand_4.4.94-user
12. halley6.v10_nand_4.4.94-userdebug
```

3. Execute make uboot-clean to clear the intermediate products generated by the last compilation:

```
$ make uboot-clean
```

4. Execute make uboot to compile:

```
$ make uboot
```

5. After the compilation is completed, the uboot image file will be generated in the project /out/product/halley6.v10_nand_4.4.94-eng/image directory, and the image can be burned directly.

3.1.2 Compile SFC Nor boot image

The boot image corresponds to the halley6_uImage_sfc_nor configuration.

1. Execute the following command in the top-level directory of the project to import the

environment variables required for compiling the halley6 platform:

```
$ source build/envsetup.sh
```

2. Execute the lunch command to select the type of image to be compiled:

```
$ lunch
```

a. If it is halley6 v10 development board, choose among the following configurations as needed:

```
13. halley6.v10_nor_4.4.94-eng
14. halley6.v10_nor_4.4.94-user
15. halley6.v10_nor_4.4.94-userdebug
```

3. Execute make uboot-clean to clear the intermediate products generated by the last compilation:

```
$ make uboot-clean
```

4. Execute make uboot to compile:

```
$ make uboot
```

5. After the compilation is completed, the uboot image file will be generated in the project /out/produc/halley6.v10_nor_4.4.94-eng/image directory, and the image can be burned directly.

3.1.3 Compile SD card boot image

Halley6 hardware does not support SD card interface yet.

3.2 u-boot source directory compilation

1. Execute the following command in the top-level directory of the project to import the environment variables required for compiling the halley6 platform:

```
$ source build/envsetup.sh
```

2. Execute the lunch command to select the type of image to be compiled. This is just to import the cross-compilation tool environment. If you only compile u-boot, you can choose at will:

```
$ lunch
```

3. Then enter the u-boot directory and execute make distclean to clear the intermediate products generated by the last compilation:

```
$ make distclean
```

Execute the make xxxxxx -jN command to compile. xxxxxx is the specific board configuration name, which needs to be determined according to the compiled startup image. For details, please refer to 2.1 boards.cfg file description. Currently, a total of 4 board-level configurations are supported, as shown in the following table:

halley6_uImage_sfc_nand	This configuration supports booting the linux kernel image uImage from nand flash
halley6_uImage_sfc_nor	This configuration supports booting the linux kernel image uImage from nor flash

halley6_xImage_sfc_nand	This configuration supports booting the linux kernel image xImage from nand flash

For example, to compile the boot image that starts uImage from nand flash, execute the following command:

```
$ make halley6_uImage_sfc_nand -jN
```

Among them, -jN is to use multi-threaded compilation, which can speed up the compilation speed. The specific N value needs to be determined according to the PC used.

4. After compiling, u-boot-with-spl-mbr-gpt.bin will be generated in the top-level directory of the u-boot project, which is the final image file and can be burned directly.

4 u-boot Application and Kernel Interface

4.1 u-boot Common commands

After entering the command line mode of uboot, enter "help" or "**?**", and then press Enter to view the commands currently supported by uboot.

Now take sfc nand flash startup as an example, the supported commands are shown in the following figure:

```
halley6# help
?       - alias for 'help'
base    - print or set address offset
boot    - boot default, i.e., run 'bootcmd'
boota   - boot android system
bootd   - boot default, i.e., run 'bootcmd'
bootm   - boot application image from memory
bootp   - boot image via network using BOOTP/TFTP protocol
chpart  - change active partition
cmp     - memory compare
coninfo - print console devices and information
cp      - memory copy
crc32   - checksum calculation
dhcp    - boot image via network using DHCP/TFTP protocol
echo    - echo args to console
env     - environment handling commands
ext2load- load binary file from a Ext2 filesystem
ext2ls  - list files in a directory (default /)
ext4load- load binary file from a Ext4 filesystem
ext4ls  - list files in a directory (default /)
fatinfo - print information about filesystem
fatload - load binary file from a dos filesystem
fatls   - list files in a directory (default /)
gettime - get timer val elapsed,
```

```
go       - start application at address 'addr'
gpio     - input/set/clear/toggle gpio pins
help     - print command description/usage
loadb    - load binary file over serial line (kermit mode)
loads    - load S-Record file over serial line
loady    - load binary file over serial line (ymodem mode)
loop     - infinite loop on address range
md       - memory display
mm       - memory modify (auto-incrementing address)
mmc      - MMC sub system
mmcinfo - display MMC info
mtdparts- define flash/nand partitions
mw       - memory write (fill)
nand     - NAND sub-system
nboot    - boot from NAND device
nm       - memory modify (constant address)
ping     - send ICMP ECHO_REQUEST to network host
printenv- print environment variables
reset    - Perform RESET of the CPU
run      - run commands in an environment variable
saveenv - save environment variables to persistent storage
setenv   - set environment variables
sfcnand - sfcnand    - SFC_NAND sub-system
```

For each command, you can also execute "help command name" to get the specific usage instructions of the command, as shown in the following figure:

```
halley6# help bootm
bootm - boot application image from memory

Usage:
bootm [addr [arg ...]]
    - boot application image stored in memory
        passing arguments 'arg ...'; when booting a Linux kernel,
        'arg' can be the address of an initrd image

Sub-commands to do part of the bootm sequence.  The sub-commands must be
issued in the order below (it's ok to not issue all sub-commands):
        start [addr [arg ...]]
        loados  - load OS image
        cmdline - OS specific command line processing/setup
        bdt     - OS specific bd_t processing
        prep    - OS specific prep before relocation or go
        go      - start OS
```

Here are some commonly used commands:

printenv	print current environment variables
setenv	Set the environment variable, the format: setenv name value ..., which means to set the name variable to the value value; if there is no such parameter, it means to delete the variable

saveenv	Save environment variables to storage device
sleep	Delay execution, format: sleep N, can delay execution for N seconds
bootm	Boot the kernel image from the specified address in memory. Format: bootm addr, the parameter is the address of the image in memory
md	Read memory data, format: md [.b, .w, .l] address [# of objects], the first parameter indicates the data unit read at one time (b: 8 bits, w: 16 bits, l: 32 bits), the second parameter address indicates the starting address of the memory to be read, and the third parameter indicates the number of data to be read from address
nand erase	Erase nand flash, format: nand erase addr1 count, the first parameter is offset, the second parameter is the number of erased bytes
nand write	Write memory data to nand flash, format: nand write addr offset count, the first parameter is the write base address, the second parameter is the offset address, and the third parameter is the number of bytes written
nand read	Read the nand flash data to the memory, the format is: nand read addr offset count, the first parameter is the read nand flash address, the second parameter is the memory location offset, and the third parameter is the number of bytes read
cp	Copy data in memory, format: cp source target count, the first parameter is the source address, the second parameter is the destination address, and the third parameter is the number of copies
cmp	Compare the data in the memory, format: cmp addr1 addr2 count, the first parameter is the memory address 1, the second parameter is the memory address 2, and the third is the comparison length (the unit is the number of bytes divided by 4, with WORDS as unit)
crc32	Calculate the check value, format: crc32 address count [addr], the first parameter is the starting address to be checked, the second parameter is the number of data bytes to check, and the third parameter is the check value saved address

4.2 u-boot Kernel parameter transfer instructions

Before the linux kernel starts, it needs to know some necessary parameters of the current hardware, such as the available memory size, the serial port device used, the device and partition where the root file system is located, etc. Regarding the above boot parameters, the linux kernel has defined a series of rules, and the bootloader responsible for starting it needs to abide by these rules to pass the boot parameters. Conveniently, u-boot has already done most of the work for us. It provides an environment variable bootargs. We only need to set the necessary boot parameters to bootargs according to the prescribed format.

Specifically, we can define CONFIG_BOOTARGS by setting the boot parameters to the macro in halley6.h.

In halley6.h, there are corresponding startup parameters according to the different file systems mounted when loading the kernel. They have some common content, which are defined as follows in

halley6.h:

```
#define BOOTARGS_COMMON "console=ttyS2,115200n8 mem=64M@0x0”
```

The specific instructions are as follows:

1. console=ttyS2, 115200 means that the serial port uses the device /dev/ttyS2 with a baud rate of 115200

2. mem=64M@0x0 means the total size of available memory is 64MB, and the starting address is 0x0

See below for specific parts of startup parameters.

4.2.1 u-boot load kernel mount network file system

The network file system is generally used for debugging during development. The current reference startup parameters in halley6.h are as follows:

```
#define CONFIG_BOOTARGS
    BOOTARGS_COMMON "ip=192.168.10.207:192.168.10.1:192.168.10.1:255.255.255.0 \
nfsroot=192.168.4.13:/home/nfsroot/fpga/user/pzqi/rootfs-tst rw"
```

The details are as follows:

1. ip=192.168.10.207:192.168.10.1:192.168.10.1:255.255.255.0 means the client (development board) IP address is 192.168.10.207, the server IP address is 192.168.10.1, and the gateway IP address is 192.168.10.1. The subnet mask ip address is 255.255.255.0.

2. nfsroot=192.168.4.13:/home/nfsroot/fpga/user/pzqi/rootfs-tst indicates that the server ip address is 192.168.4.13, and the mounted server directory is /home/nfsroot/fpga/user/pzqi /rootfs-tst.

3. rw indicates that the mounted file system is readable and writable.

Note: If the kernel is configured with dual network controllers, you need to specify the device that starts the network file system in the ip item:

ip=192.168.10.207:192.168.10.1:192.168.10.1:255.255.255.0::eth0:off /*According to the selected network port, modify eth0:off or eth1:off*/.

4.2.2 u-boot Load the kernel to mount the jffs2 filesystem

This startup parameter corresponds to the **halley6_uImage_sfc_nor** configuration. The current startup parameters in halley6.h are as follows::

```
#define CONFIG_BOOTARGS
    BOOTARGS_COMMON "ip=off init=/linuxrc rootfstype=jffs2 root=/dev/mtdblock2 rw flashtype=nor"
```

The specific instructions are as follows:

1. ip=off means do not use the network file system

2. init=/linuxrc indicates that the init process uses linuxrc in the root directory

4. rootfstype=jffs2 indicates that the root file system format is jffs2

5. root=/dev/mtdblock2 indicates that the device storing the root file system is /dev/mtdblock2, which corresponds to the partition numbered 2 in nor flash

6. rw indicates that the mounted file system is readable and writable.

4.2.3 u-boot Load the kernel to mount the ubifs file system

This startup parameter corresponds to the **hallry6_uImage_sfc_nand** configuration. The current startup parameters in halley6.h are as follows:

```
#define CONFIG_BOOTARGS
    BOOTARGS_COMMON "ip=off init=/linuxrc ubi.mtd=2 root=ubi0:rootfs ubi.mtd=3 rootfstype=ubifs rw
flashtype=nand"
```

The specific instructions are as follows:

1. ip=off means do not use the network file system to start
2. init=/linuxrc indicates that the init process uses linuxrc in the root directory
3. ubi.mtd=2 root=ubi0:rootfs ubi.mtd=3 rootfstype=ubifs rw indicates that the current NAND flash partitions numbered 2 and 3 use the ubifs file system, and the root file system is mounted in a readable and writable manner loaded on partition numbered 2.

4.3 u-boot load the kernel image

u-boot loads the kernel image through tftp or fastboot, which saves the process of burning the kernel image and directly loads the kernel image into the memory. Using this method is very convenient for debugging during the development process.

4.3.1 Uboot Load the kernel image via tftp

The tftp method can directly download the kernel image to the memory through the network, so please note that the use of this method requires u-boot to support the network.

1. uboot only supports one gmac controller to work, you need to configure the selected controller and the interface mode of the corresponding controller in halley6.h:

```
/* Select GMAC Controller */
#define CONFIG_GMAC0 /* According to the actual test, configure the corresponding GMAC1, GMAC0*/
```

2. PC server configuration

```
$ sudo apt-get install tftpd-hpa #Install the server program
$ sudo service tftpd-hpa start  # Start the server
$ sudo apt-get install tftp-hpa  # Install the client program
$ tftp 127.0.0.1                 # Test whether the server is set up successfully tftp> get test.txt
```

3. Development board configuration

a. Enter the uboot command line mode and set the ip address according to the network conditions:

```
halley6# set ipaddr 192.168.4.145
halley6# set serverip 192.168.4.146
```

b. Put the required uImage into the PC server folder, and execute the following command to download the uImage to the memory location 0x80800000:

```
halley6# tftp 0x80800000 uImage
```

c. start uImage:

```
halley6# bootm 0x80800000
```

4.3.2 Uboot loads the kernel image through the fastboot command

The fastboot method is to use the usb port to download the kernel image to the memory and start the command. It is only used for debugging and does not support other functions of android fastboot.

1. Install the fastboot application on the PC side using the following command:

```
$ sudo apt-get install android-tools-fastboot
```

2. Enter the uboot command line mode and enter the fastboot command:

```
halley6# fastboot
```

3. Execute the following commands in the path where the kernel image is stored on the PC:

```
$ sudo fastboot boot uImage
```

4. After the command is executed successfully, the kernel image can be downloaded to the memory and started.

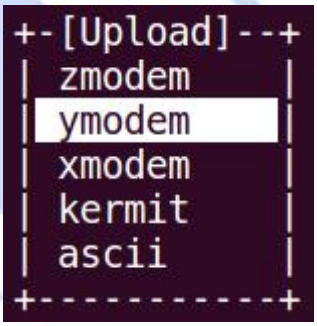
4.3.3 u-boot Load the kernel image through the serial port

The serial port method is to directly download the kernel image to the memory through the serial port. Note that the transmission speed of this method will be slower, please choose according to the actual situation.

1. Enter the u-boot command line mode and execute the following command:

```
halley6# loady 0x80800000 115200
## Ready for binary (ymodem) download to 0x80800000 at 115200 bps...
C
```

2. First hold down the ctrl key, then hold down the a key, then release the two keys at the same time, and finally press the s key, you can see the screen as shown below:

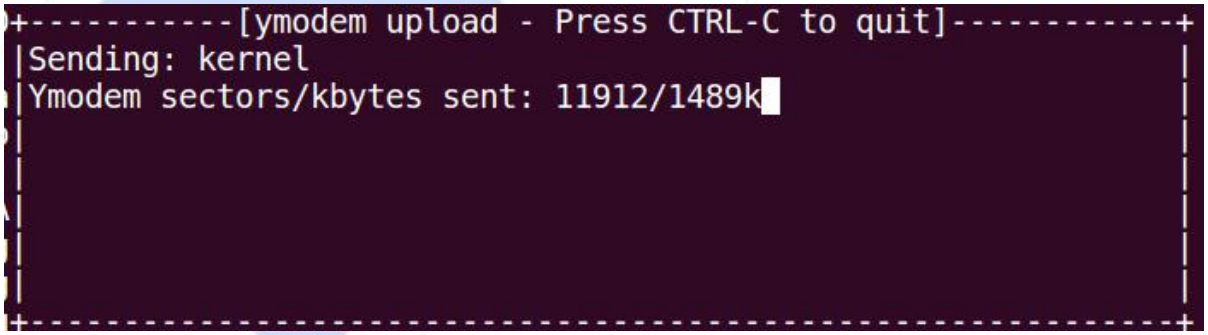


Here press the ↓ key to select the ymodem protocol, and then press Enter ;

Then the screen as shown below will appear, where you can select the file to be transferred, press the ↑ ↓ key to move, double-tap the space bar to enter the directory, single-tap the space bar to select the file, and press the Enter key to confirm the selection.

We choose the kernel image that needs to be transferred:

3. After the selection is complete and press Enter to confirm, the following screen for file transfer will appear:



4. After the transfer is completed, you can start the kernel image written to the memory location 0x80800000 through the bootm command:

```
$ bootm 0x80800000
```

5 Ingenic tools introduce

Ingenic tools is a collection of tools added by Ingenic manufacturers in u-boot.

5.1 DDR parameter related

Source location:u-boot/tools/ingenic-tools/

```
├──── ddr_creator_x1600
│     ├──── add_chip_info.c
│     ├──── ddr3_params.c
│     ├──── ddr_params_creator.c
│     ├──── ddr_params_creator.h
│     ├──── lpddr2_params.c
│     ├──── lpddr3_params.c
│     ├──── Makefile
│     ├──── supported_ddr_chips.c
```

This is mainly used to generate the corresponding DDR parameters, and then write to the specified location of the u-boot image to distinguish.

5.2 MBR/GPT Partition Table

Source location:u-boot/tools/ingenic-tools/

```
├──── gpt.bin
├──── gpt_creator.c
├──── gpt_tab_to_c.sh
├──── mbr_creator.c
├──── mbr-of-gpt.bin
├──── mk-gpt-xboot.sh
```

Both GPT and MBR are standards for hard disk partitioning.

MBR: Master Boot Record, the main partition boot record, records the relevant information of the hard disk itself and the size and location information of each partition of the hard disk.

GPT: Globally Unique Identifier Partition Table, GUID partition table, is a newer disk partition table structure standard derived from the UEFI standard. Compared with the MBR partition scheme, GPT provides a more flexible disk partitioning mechanism.

The role of the file here is to generate the corresponding partition data according to the standards of GPT and MBR.

5.3 nand/nor flash related

Source location:u-boot/tools/ingenic-tools/

1. Save the hardware parameters of various nand flashes for the sfc driver in spl

```
├──── nand_device
```

2. Save the partition parameters of nand flash and nor flash, which is different from the partition parameters written by the burning tool, which will eventually be read by the sfc driver in the kernel and generate the corresponding mtd partition, which can be changed if necessary.

```
├── sfc_builtin_params
│   ├── nand_device.c
│   ├── nand_device.h
│   ├── nor_device.c
│   └── nor_device.h
```

3. Write the partition parameters of nand flash and nor flash to the specified location in the u-boot image.

```
├── sfc_nand_builtin_params.c
├── sfc_nor_builtin_params.c
```

5.4 other

Source location:u-boot/tools/ingenic-tools/

1. The hardware timing parameters used to generate the sfc controller

```
├── sfc_timing_params.c
```

2. When used for sfc boot, the header data in front of spl is generated, and spl is loaded for bootom for verification purposes

```
└── spi_checksum.c
```

6 Reference documents

1. u-boot/README
2. kernel-4.4.94/Documentation/kernel-parameters.txt